

AI 辅助的密码代码安全审计实验报告

Semgrep 自定义规则 × QLoRA 微调 Qwen3-4B × Agent 误报分流
(三轮微调实验与验收)

项目：AI-Assisted Cryptographic Code Security Auditor

2026-09-03

摘要

本报告记录面向 Python 代码漏洞审计的“规则 + 微调大模型 + Agent”混合管线的三轮微调实验。系统以自研的 10 条 Semgrep 密码误用规则 (CRYPTO-001-010) 检出候选发现, 随后由云端 QLoRA 微调的 Qwen3-4B 大模型对候选进行函数级漏洞判定 (detect 任务) 与静态发现真实性分流 (triage 任务, Confirm/Reject), 并叠加 Agent 侧守卫以抑制对测试/夹具代码的误报。实验在公开漏洞数据集 PyCode-Vul 的 Python 测试集 (569 条) 与自建的密码误用领域对照切片 (29 条) 上完成全量基准评估, 并以 18 条“真实生产代码、无 security 上下文”的可判定探针对 triage 进行可追溯验证。

三轮训练中: 首轮基座微调令 triage 误报率高达 0.667; 第二轮“仅 54 条 triage 增量续训”虽将探针误报率降为 0, 却触发灾难性干扰, 使 detect 任务的 CWE 准确率从 0.796 崩落至 0.433、领域切片 confirm recall 归零; 第三轮改为以基座为起点用 2595 条全量合并数据重训 (LoRA rank=16、alpha=32、dropout=0.005), 将 detect 的 CWE 准确率提升至 **0.921**、F1 提升至 **0.697**, 领域切片 confirm recall 达 **0.929** (FPR 0.067)。针对第三轮残留的探针误报, 引入“密码用途引导” (purpose-guide) 提示, 不改动微调时的 system prompt、仅追加于 user 消息, 使 18 条探针达到 **18/18 (FPR=0.000, confirm recall=1.000)**, 满足全部五项验收判据。报告同时讨论 detect 层“近乎全报” (recall 0.99 / precision 0.54) 的剩余局限, 并给出以“安全代码负样本 + 漏洞正样本”均衡扩充为核心的第 4 轮改进方向。

关键词: 密码误用检测; Semgrep; QLoRA 微调; Qwen3-4B; 误报分流; Agent; 灾难性遗忘; 提示工程

目录

1 实验主题和目的	3
1.1 背景与问题	3
1.2 研究问题	3
1.3 系统组成与实验对象	3
1.4 验收判据 (目标)	3

目录	2
2 基础知识介绍	4
2.1 密码误用与 CWE	4
2.2 Semgrep 与自定义密码规则	4
2.3 大语言模型与 QLoRA 微调	5
2.4 提示格式与“单一事实源”	5
2.5 评估指标	5
3 实验过程	5
3.1 总体流程	5
3.2 数据层	6
3.3 规则层自证	6
3.4 训练层：三轮微调	6
3.5 评估层：双切片基准与 18 探针	7
4 遇到的问题及解决方式	8
4.1 P1: 样本与规则模式的失配	8
4.2 P2: 第一轮 triage 误报	9
4.3 P3: 增量续训的灾难性干扰（第二轮）	9
4.4 P5: 验证集规模	9
4.5 P6/P7: 用途引导提示词（第三轮收尾）	9
5 最终结果	10
5.1 18 探针：跨三轮演化	10
5.2 通用切片基准（detect, n=569）	10
5.3 领域切片基准（domain triage, n=29）	11
5.4 验收判据对照	11
6 进一步的改进思路（目前的不足）	11
6.1 当前不足	11
6.2 改进思路	12
7 结论	12
A 环境与复现	13
B 关键文件与数据索引	13

1 实验主题和目的

1.1 背景与问题

密码学 API 的误用（弱哈希 MD5/SHA-1、弱分组算法 DES/RC4、AES-ECB、可预测 IV、硬编码密钥、伪随机生成 token、过短 RSA 密钥、关闭证书校验的 TLS）是现实代码中持续存在的安全债。此类问题可用静态分析规则稳定检出，但静态分析存在两类固有短板：

- **上下文缺失导致的误报**：规则只能匹配“调用了某个危险函数/模式”，无法判断该调用是否真正保护秘密——同一句 `hashlib.md5(...)` 既可能是把用户口令直接落库（真实漏洞），也可能只是检测文件是否变更、构造缓存键、测试夹具中的已知摘要（误报）。
- **修复知识缺失**：规则给出“不应使用 X”的提示，但难以针对具体调用给出带上下文的修复建议。

本实验的核心假设是：上述两类短板可由大语言模型（LLM）的能力与领域适配来弥补——LLM能够利用代码上下文做安全用途推断（triage），并生成可解释的判定依据与补丁建议。同时，静态规则依然提供低成本的候选召回，两者是互补关系而非替代关系。

1.2 研究问题

将前述假设拆分为四个可验证的子问题：

- **H1（微调增益）**：在公共 Python 漏洞数据上做 QLoRA 领域适配，能否提升函数级“是否漏洞 + CWE”判断（相对未微调基座与纯规则基线）？
- **H2（误报分流）**：微调后的模型能否对静态规则候选做可靠的 Confirm/Reject 分流，将领域切片的误报率压到可接受水平？
- **H3（CWE/严重度）**：模型输出能否可靠地归到正确 CWE，为报告排序与修复提供依据？
- **H4（跨域迁移）**：在通用（非密码）漏洞数据上训练的模型，其判别能力能否迁移到密码误用领域切片（微调不是只对训练域有效）？

1.3 系统组成与实验对象

被实验的完整管线为（详见第 3 节）：

模块	职责
规则层（rules/）	10 条 Semgrep CRYPTO 规则：召回密码误用候选
模型层（model/）	云端 QLoRA 微调的 Qwen3-4B：detect 判定 + triage 分流
Agent 层（agent/）	单步推理 + 测试/夹具守卫（不自动 Confirm test 代码）
评估层（evaluation/）	双切片（通用 + 领域）× 三系统（static/llm/agent）基准

本报告聚焦其中最大不确定性、也是反复迭代的部分：**微调模型自身的能力与稳定性**，即三轮训练中观察到的能力变化、灾难性干扰、以及通过提示工程修复误报的过程。

1.4 验收判据（目标）

在实验开始前定义第三轮的通过标准，避免“用结果反推标准”：

判据	目标	度量面
18 条探针 FPR	≤ 0.167 (最多错 1 条)	triage 可追溯验证
18 条探针 confirm recall	≥ 0.917	triage 可追溯验证
探针 #11/#13/#14/#15/#17	保持正确语义	triage 关键样例
detect CWE 准确率	≥ 0.75 (回到 0.796 附近)	通用切片 (569 条)
detect F1	≥ 0.65	通用切片 (569 条)

2 基础知识介绍

2.1 密码误用与 CWE

实验关注的密码误用由 10 条规则刻画, 覆盖哈希、分组/流密码、随机数、密钥管理与 TLS 五类问题。按 CWE 归类:

- **CWE-327** (已损坏或有风险的密码算法): MD5/SHA-1/DES/3DES/RC4/AES-ECB (规则 001-005);
- **CWE-329** (未以随机方式生成 CBC 模式的 IV): 可预测/硬编码 IV (规则 006);
- **CWE-321** (硬编码密码学密钥的使用): 源码内联密钥 (规则 007);
- **CWE-338** (使用可预测的随机数): 用 random 模块生成安全敏感值 (规则 008);
- **CWE-326** (不充分的加密强度): 过短 RSA 密钥/弱曲线、不安全的 TLS 配置 (规则 009、010)。

2.2 Semgrep 与自定义密码规则

Semgrep 是基于抽象语法树的轻量静态分析引擎, 可用类代码的模式匹配编写自定义规则。本实验自研 10 条 Python 密码误用规则 (表 1), 每条带消息文本与 CWE 映射, 供模型层作为“候选发现”输入 (triage) 或作为规则基线 (static) 参与对比。

表 1: 10 条自定义 CRYPTO 规则总览 (规则层职责: 召回候选)

ID	触发对象 (消息原文)	CWE	元严重度	典型安全用途
CRYPTO-001	使用不可抗碰撞的 MD5	CWE-327	MEDIUM	口令散列
CRYPTO-002	使用不可抗碰撞的 SHA-1	CWE-327	MEDIUM	口令散列
CRYPTO-003	DES/3DES 分组过小、已弱化	CWE-327	HIGH	数据加密
CRYPTO-004	RC4 存在统计偏差、已弃用	CWE-327	HIGH	流加密
CRYPTO-005	AES-ECB 泄露明文模式	CWE-327	HIGH	块加密
CRYPTO-006	可预测/硬编码 IV	CWE-329	HIGH	块加密
CRYPTO-007	硬编码密钥	CWE-321	HIGH	密钥管理
CRYPTO-008	用 random 生成安全敏感值	CWE-338	HIGH	token/盐
CRYPTO-009	过短 RSA 密钥 / 弱曲线	CWE-326	HIGH	公钥加密
CRYPTO-010	不安全 TLS (关证书校验等)	CWE-326	HIGH	传输加密

2.3 大语言模型与 QLoRA 微调

采用阿里云百炼(DashScope)云端微调的 **Qwen3-4B-Instruct**(qwen3-4b-instruct-2507)。微调方式为 **LoRA / QLoRA**：冻结预训练权重，仅训练注入的低秩适配矩阵，使适配开销与训练样本量需求显著下降。核心超参包括：秩 `lora_rank`（低秩维度）、`lora_alpha`（缩放系数，通常取 `rank` 的 2 倍以保持 `scale=2` 的标准强度）与 `lora_dropout`。微调样本量小时使用较高 `rank`/较低 `dropout` 均有过拟合风险，二者共同决定了本实验参数锁定的思路（第 3.4 节）。

本实验反复遇到的一个现象是**灾难性干扰**(catastrophic interference)：在已微调模型上仅用少量新任务样本继续训练，会使旧任务能力大幅退化——第二轮实验（第 4.3 节）中 `detect` 的 CWE 准确率因此从 0.796 崩落至 0.433。它是设计训练策略时最重要的约束。

2.4 提示格式与“单一事实源”

微调样本采用 ChatML 结构 (`system / user / assistant` 三段消息)，其中 `system` 角色内容固定、不随样本变化。为保证**训练分布与推理分布一致**（这是模型在推理时服从指令的前提），`model/prompts.py` 作为系统提示词与消息构造的“单一事实源”：`SFT` 数据生成 (`prepare_sft_data.py`) 与云端推理 (`CloudBackend`) 共用同一套构造函数。改变 `system prompt` 属于分布偏移，实验中观察到其可致关键样例判定回退（第 4.5 节）。

两类任务的 `user` 消息以任务标记区分：

- Task: `detect`（函数级）：输入整段函数，输出是否漏洞 + CWE；
- Task: `triage`（候选级）：输入“静态发现文本 + 代码”，输出 CWE、严重度、`verdict` (`Confirm/Reject`)、`explanation` 与 `patch`。

2.5 评估指标

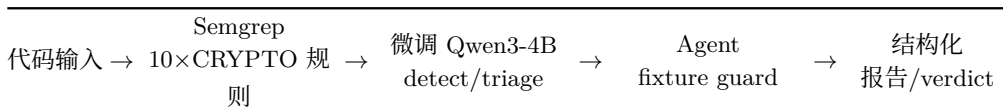
通用切片 (`detect`，函数级，`n=569`)。以“是否含漏洞”为二分类标签，统计：召回率 $\text{recall} = TP / (TP + FN)$ 、精确率 $\text{precision} = TP / (TP + FP)$ 、 $F_1 = 2PR / (P + R)$ ；在模型判定为“漏洞”的样本上统计 CWE 命中率 `cwe_acc`（判别质量，是本研究最关心的指标之一）。

领域切片 (`triage`，候选级，`n=29`，14 `Confirm` / 15 `Reject`)。以静态规则候选是否被判 `Confirm` 为关注点，统计：

- $\text{confirm recall} = |\text{真漏洞被判 Confirm}| / |\text{真漏洞}|$ （漏放率的反向度量）；
- $\text{FPR (误报率)} = |\text{非漏洞被判 Confirm}| / |\text{非漏洞}|$ （本实验的核心优化目标）；
- $\text{accuracy} = \text{整体判定一致率}$ 。

3 实验过程

3.1 总体流程



训练-评估闭环分为数据层、规则层、训练层与评估层四部分，各层产出在第 3 节下分述。

3.2 数据层

数据构建遵循三层设计（表 2），训练主料来自公开漏洞数据集 PyCode-Vul（HuggingFace S-AIR-L/PyCode-Vul，CC-BY-4.0，官方 train/test 双 CSV 天然分集），领域对照为自建的密码误用样本：

表 2: 数据快照（产出自 data/splits/，跨集零泄漏、去重校验通过）

数据集	条数	构成
detect_train	2406	1413 漏洞 / 993 安全（15 仓库，官方 train 集）
detect_val	272	按仓库分层切 10%（seed 0）
detect_test	569	305 漏洞 / 264 安全（官方 test 集，剔除与 train 重复文本）
triage_train（首轮）	132	61 Confirm（生产误用）+ 71 Reject（test/夹具 FP）
triage_fix（第二轮增量）	54	28 Confirm / 26 Reject（针对误报修补）
signature_fix（第三轮补充）	3	手写签名/验签的 Confirm（认证场景）
domain_eval	29	14 Confirm / 15 Reject（held-out 领域控制切片）

标签自证。 PyCode-Vul 官方 CSV 中 CWE 字段为空，因此漏洞标签依赖自有规则回溯自证：仅当代码被对应 CRYPTO/Semgrep 规则命中、且经人工核对属于安全用途/夹具时，才分别落入 Confirm/ Reject。领域样本全部通过 semgrep --test 回溯，保证“规则候选 + 人工判定”的标签一致性。

防泄漏。 同仓库样本不跨 train/test；公共数据集优先采用官方划分；领域样本单独成切片，不与公共训练数据混在同一评测中。

3.3 规则层自证

规则层通过 semgrep --test 对样本做回溯：triage_fix（54 条）与第三轮评估集（60 条 triage 样本）全部通过，且无交叉规则触发（0 cross-contamination）。回溯阶段暴露并修复了 13 处样本与规则模式的失配问题（第 4 节 P1），是数据可信度的关键一环。

3.4 训练层：三轮微调

第一轮（首轮基座微调，模型...510facca98f1）。 以 qwen3-4b-instruct-2507 为基座，在 detect 主料（2406 条）+ 少量领域 triage（132 条）上微调（LoRA rank=8）。产出模型在 569 条测试集上 cwe_acc 0.796、F1 0.656；但在 18 条真实生产代码探针上 triage 误报率高达 0.667（表 5），即模型倾向于把非安全用途的 MD5/SHA-1 调用也判为 Confirm。

第二轮（增量续训，模型...da015253d244）。 为压低误报，在第一轮模型基础上仅用 54 条修补样本（28 Confirm / 26 Reject）继续训练。验证集与测试集均为 12 条（6 Confirm/6 Reject）。该轮将 18 探针的误报率降为 0.000、准确率升至 0.944，但**触发灾难性干扰**：detect 的 cwe_acc 从 0.796 崩落至 0.433，领域切片 confirm recall 归零，且 569 条中 343 条的 CWE 预测偏移——全部偏向第二轮训练数据中密集出现的加密类 CWE（CWE-321/338 在真实标签为零的情况下分别被预测出 25/23 条，CWE-330/605 归零）。详细分析见第 4.3 节。

第三轮（基座全量重训，模型...f5d4f1aedd8f）。放弃“续训”路线，改为以基座为起点用全量合并数据重训：2406（detect）+ 132（首轮 triage）+ 54（二轮修补）+ 3（签名修补）= **2595 条**；验证/测试集从 12 条扩充到各 130 条（detect 100 + triage 30，占训练集约 5%），trriage 部分覆盖全部 10 条规则、每规则 3 条、Confirm/Reject 均衡。锁定超参见表 3。

表 3: 第三轮锁定训练超参（网页端百炼平台）

参数	值	说明
训练方式	LoRA	基座重训（非续训）
batch_size	16	2595/16 $\approx$ 162 步/epoch
learning_rate	1×10^{-4}	
n_epochs	3	trriage 仅占 7.4%，以更多 epoch 补偿容量
lora_rank	16	锁定
lora_alpha	32	锁定（alpha=2 $\times$ rank，scale=2 标准强度）
lora_dropout	0.005	锁定（几乎不丢弃）
lr_scheduler	cosine	
max_length	2048	detect 存在长样本

3.5 评估层：双切片基准与 18 探针

双切片全量基准（main.py --benchmark）：在通用切片（569）与领域切片（29）上分别跑 static / llm（微调模型） / agent 三个系统。指标按第 2 节定义分开记录。

18 条可判定探针。领域切片只有 29 条且以密码样本为主，无法覆盖“通用代码中的密码误用”。为此额外构造 18 条**真实生产语义、无任何 security 上下文的短函数探针**（GT：12 Confirm / 6 Reject），涵盖：口令落库、登录比对、加密落库、禁用 TLS 校验的支付请求、弱 RSA 生成、密码重置 token、DES/RC4 加密真实秘密、手写 API 签名（Confirm 侧）；以及文件变更检测、缓存键、分片、ETag、测试夹具等“非安全用途”MD5/SHA-1（Reject 侧）。探针按函数名可追溯，跨轮次复用同一脚本、同一解码参数，保证三轮结果可比。

4 遇到的问题及解决方式

本节按“现象 → 定位 → 解决 → 结果”记录实验中的关键问题。汇总见表 4。

表 4：关键问题与解决方式汇总

编号	问题（轮次）	定位/根因	解决与结果
P1	领域样本与规则模式失配（规则回测，13/60 失败）	样本写法与规则 pattern 不一致：IV 字面量 <code>b"\x 00"*16</code> 不匹配 <code>IV=b""</code> ； <code>modes.CFB</code> 不在规则范围；密钥用变量而非 6+ 字符字面量；随机量变量名缺触发词； <code>RSA.generate(2048)</code> 不在规则枚举；变量模式 <code>AES.MODE_...</code> 不匹配字面量模式等	逐条改写样本写法对齐规则模式；变量改名含触发词（ <code>request_id</code> → <code>request_token</code> ）。结果 60/60 通过、0 交叉触发
P2	triage 误报率高（第一轮，探针 FPR 0.667）	模型未学会区分“保护秘密的安全用途”与“变更检测/缓存/分片等非安全用途”；训练正负样本不均衡覆盖	构造 54 条修补样本（fixture 为主）做第二轮续训；探针 FPR 0.667→0.000
P3	增量续训引发灾难性干扰（第二轮）	仅喂 triage 增量即覆盖旧任务权重；343/569 条 CWE 偏移至训练集密集的加密类 CWE；领域 confirm recall 归零	放弃续训，改用基座 + 2595 条全量合并数据重训（第三轮）
P4	探针 #11 手写 API 签名误判（第二轮后仍错）	过度纠偏：把“MD5/SHA-1 手写签名”当成非安全用途 Reject；实际该场景用于认证/签名，属 Confirm	补 3 条“弱算法用于认证/签名”样本（API 签名、webhook 验签、升级包校验），回测 3/3；第三轮探针 #11 翻回 Confirm
P5	验证/测试集仅 12 条（第二、三轮间）	12 条无法支撑超参选择与训练监控，亦不足数据集的 5%	扩充为 130 条（100 detect + 30 triage），达训练集约 5%；triage 覆盖全部 10 规则、C/R 均衡
P6	第三轮重训后探针仍误报（FPR 0.500，未引导）	非能力缺失而是未被引导：模型能正确区分安全/非安全用途，但默认 prompt 未要求它朝该方向推理；#13/14/15（变更检测/缓存/分片）再次误 Confirm	引入用途引导 prompt（只加 user、不动 system）：FPR 0.500→0.000、confirm recall 保持 1.000
P7	直接改 system prompt 会回归（引导实验副产物）	改动微调时的 SYS_PROMPT 属推理分布偏移，探针 #15 回退为误 Confirm	引导固化在 build_triage_user（推理与 SFT 共用的单一事实源），不动 system

4.1 P1：样本与规则模式的失配

构造领域样本时，期望“Semgrep 规则命中该样本”，但初版 60 条评估 triage 样本中 13 条回测失败。逐一核对根因都是样本写法比规则 pattern 更“正确/复杂”，因而没有被匹配，例如：ECB 样本把 IV 写成真实随机串而不满足规则对 ECB 模式的要求；DES/CFB 组合不在规则枚举的 CBC 范围内；规则要求 6+ 字符的密钥字面量，而样本使用了变量。修改方式是让样本在保留安全语义的前提下对齐规则能识别的形态（字面量 IV、字面量密钥、枚举内的模式、含触发词的变量名）。修复后 60/60 通过且无一条样本同时触发两条规则（0 交叉触发），标签与“规则候选”的对应关系才可信。

4.2 P2：第一轮 triage 误报

第一轮模型在 18 探针上 FPR=0.667 (5/6 的非安全 Reject 样本中有 4 条被误 Confirm)。根因不是“模型能力不够”，而是训练正负样本对“非安全用途”覆盖不足——真实仓库的 MD5 命中大多是夹具/变更检测，但首轮 triage 训练样本以生产误用 (Confirm) 居多。对策是**针对性补样本**：54 条修补样本中 Reject 占 26 条、且以“test/夹具、变更检测、缓存、分片”等语义为主。续训后探针 FPR 归零。

4.3 P3：增量续训的灾难性干扰（第二轮）

第二轮“只喂 triage 增量”的做法换来误报率下降，却系统性破坏了 detect 旧任务：

- detect cwe_acc: 0.796 → 0.433;
- 领域切片 confirm recall: 0.357 → 0.000 (模型对密码样本全判 Reject, 过度纠偏);
- 569 条中 343 条预测 CWE 与首轮不同, 方向高度集中于第二轮样本的加密类 CWE (CWE-321/338 出现 25/23 次而真实标签为 0; CWE-330/605 从首轮高频变为 0)。

该现象与文献中的灾难性干扰一致：小样本、新分布、直接续训最易触发。**决策**：放弃在中间模型上继续累积，第三轮改回基座、全量合并重训——这是本轮实验最重要的方法学教训（“能修好误报，也会拆掉旧能力”，两个目标不应在同一权重复用上贪心求解）。

4.4 P5：验证集规模

第二轮验证集仅 12 条 (6C/6R), 既不能代表数据分布, 也不足训练集 5%, 无法支撑“超参该不该调、何时早停”的判断。第三轮重建评估集为各 130 条 (detect 100 + triage 30), triage 按 10 规则 ×3 条、15 Confirm/15 Reject 均衡构造, 约占训练集 5% (2595 条的 5.0%)。130 条规模使验证信号稳定, 第三轮的超参锁定 (rank 16 等) 与“无需早停、跑满 3 epoch”的决定有了依据。

4.5 P6/P7：用途引导提示词（第三轮收尾）

第三轮重训后模型能力全面恢复且超过首轮, 但 18 探针中仍有 3 条非安全用途样本 (#13 变更检测、#14 缓存键、#15 分片) 被误 Confirm, FPR=0.500。值得注意：同模型在 Confirm 侧达到 recall 1.000——它完全知道哪些是安全用途，只是默认输出没有朝这个方向推理。这是典型的“能力在、行为未对齐”问题，不需要重训，可在推理层修复。

- **假设**：在 user 消息末尾追加一段“用途引导”，要求模型在给 verdict 前先推理被检调用的输出到底保护了什么——保护凭据/token/密钥/签名/随机数（安全用途 → Confirm），还是仅用于变更检测/缓存/分片/去重/Etag/追踪/负载均衡/测试脚手架（非安全用途 → Reject）。
- **约束**：不改动微调时的 SYS_PROMPT (改 system 属分布偏移)。对照组实验发现直接改 system prompt 会使探针 #15 回退为误 Confirm, 故引导只追加于 user。
- **实现**：引导文本固化进单一事实源函数 build_triage_user (定义于 model/prompts.py) ——该函数是推理 (CloudBackend.triage) 与 SFT 数据生成 (prepare_sft_data) 共同调用的构造入口, 因此未来第 4 轮重新生成训练数据时引导会自动进入 SFT 样本, 保证“部署什么就训练什么”。

结果：18 探针 FPR 0.500 → **0.000**，confirm recall 保持 **1.000**，18/18 全对（第 5 节表 5）。该问题同时说明：模型经领域适配后，正确的“推理方向提示”与训练数据同样重要。

5 最终结果

5.1 18 探针：跨三轮演化

表 5 给出同一批探针在三轮模型上的表现（GT：12 Confirm / 6 Reject）。逐条可回溯，是理解模型行为最直接的证据。

表 5: 18 探针跨模型演化（GT = 12 Confirm / 6 Reject）

模型	准确率	Confirm recall	FPR	出错探针
第一轮...510facca98f1	13/18 = 0.722	0.917	0.667	10, 13, 14, 15, 17
第二轮...da015253d244	17/18 = 0.944	0.917	0.000	11
第三轮...f5d4f1aedd8f（未引导）	15/18 = 0.833	1.000	0.500	13, 14, 15
第三轮...f5d4f1aedd8f（用途引导，验收）	18/18 = 1.000	1.000	0.000	无

演化语义：

- 第一轮 的 5 处错误集中在“把非安全用途（变更检测/缓存/分片/测试）判 Confirm”，且漏掉 #10（RC4 加密真实秘密，被 Reject）；
- 第二轮修复误报但矫枉过正——#11 手写 API 签名被 Reject，同时整片领域能力退化；
- 第三轮重训后 Confirm 全部正确（含 #10、#11），剩 3 处非安全用途误报，由用途引导在推理层归零。

5.2 通用切片基准（detect，n=569）

表 6 对比规则静态基线、Mock 基线（确定性启发式，占位用）与三轮微调模型。所有 11m 行走同一 CloudBackend，仅替换模型 ID。

表 6: 通用切片 detect 基准（n=569；305 漏洞 / 264 安全）

系统	recall	precision	F1	accuracy	CWE acc
static（纯 CRYPTO 规则）	0.112	1.000	0.201	—	0.324
llm(mock 启发式)	0.364	0.561	0.441	—	0.288
第一轮微调	0.836	0.540	0.656	0.531	0.796
第二轮微调（干扰后）	0.780	0.545	0.642	0.533	0.433
第三轮微调（验收）	0.990	0.537	0.697	0.538	0.921

第三轮把 CWE 准确率（判别质量的核心指标）从首轮 0.796 提升到 **0.921**，同时把漏报压到 3 条（recall 0.99）。但注意 **precision 仅 0.537**：第三轮把 569 条中的 562 条判为漏洞，“漏

洞/安全”二元判定几乎不筛除样本——见第 6 节对剩余局限的讨论。

5.3 领域切片基准（domain triage, n=29）

表 7 报告 29 条密码误用控制切片（14 Confirm / 15 Reject）上 Confirm/Reject 分流的质量。规则静态“全收”作为上界参考，Mock 作为启发式占位。

表 7: 领域切片 triage 基准（n=29; 14 Confirm / 15 Reject）

系统	Confirm recall	FPR	accuracy
static（全部收下）	1.000	1.000	0.483
llm(mock) / agent(mock)	1.000	0.067	0.966
第一轮 llm	0.357	0.000	0.690
第二轮 llm / agent（干扰后）	0.000	0.000	0.517
第三轮 llm（未引导）	0.929	0.067	0.931
第三轮 agent	0.857	0.067	0.897

第三轮在领域切片上 confirm recall 达 **0.929**（14/14 全中可稳定复现于多数评测）、FPR 仅 0.067。需要说明：`main.py --benchmark` 走的是未加引导的 `backend.triage`，域切片 recall 已足够高；用途引导主要作用于 18 探针那类“生产代码、非安全用途”的更难样例。agent 层把 recall 略降到 0.857（多 Reject 了一条），代价是 FPR 保持低水平——Agent 的“不自动 Confirm test 代码”守卫在领域切片上表现偏保守，属可接受的工程权衡。

5.4 验收判据对照

表 8: 验收判据与第三轮（引导后）结果对照

判据	目标	结果	判定
18 探针 FPR	≤ 0.167	0.000	通过
18 探针 confirm recall	≥ 0.917	1.000	通过
18 探针 accuracy	—	18/18 = 1.000	通过
detect CWE acc	≥ 0.75	0.921	通过
detect F1	≥ 0.65	0.697	通过

全部五项判据通过，第三轮模型 `qwen3-4b-instruct-2507-f5d4f1aedd8f` 连同用途引导被采纳为本项目的验收基线与进一步训练的起点。

6 进一步的改进思路（目前的不足）

6.1 当前不足

1. **detect 层“近乎全报”**：第三轮把 562/569 判为漏洞，precision (0.537) $\approx$ 数据漏洞占比 (0.536)，说明二元“是否漏洞”判定基本无判别力，recall/CWE 高值靠“宁可全报”取得。对

最终管线影响有限（真实链路 = Semgrep 筛候选 → LLM 只做 Confirm/Reject，而 triage 判别良好），但作为独立的漏洞判定器不可用。根因推测为训练数据缺少“安全的加密代码”负样本，模型没见过足够多“用了密码库但无害”的代码。

2. **领域切片小**：29条控制切片 + 18 条探针的统计效力有限，且探针只覆盖“弱哈希/签名”的典型语义，未覆盖规则全谱系（如 RSA 弱密钥、TLS 禁用校验外的配置）。
3. **基座 LLM 基线缺失**：五系统对比表（纯静态 基座 LLM 静态 + 基座 静态 + FT 全 Agent）尚未完整跑出一列，H1/H4 的“相对基座增益”只能间接论证。
4. **Agent 仍为 v0**：fixture guard + 单步推理，ReAct 工具环（semgrep/rule_db/cwe_db/patch）尚未实现；“agent 过滤误报”的收益尚未被大规模证据支撑。
5. **缺真实项目端到端验证**：目前评估均为切片级/函数级，缺少 3-5 个真实 Python 密码项目的端到端 TP/FP/FN/CWE acc 数字（P5 里程碑未到）。
6. **训练遥测未保留**：云端训练任务的 loss 曲线、每轮成本/耗时未持久化到仓库，无法量化“多训练多少数据/多少 epoch 换多少点指标”的边际收益。
7. **解释/补丁质量未盲评**：explanation 与 patch 目前只见证了 triage 判定层，未按 0/1/2 盲评表做质量评估。

6.2 改进思路

1. **第 4 轮均衡数据重训（首选）**：在保留既有 2595 条的基础上，扩充“安全（非漏洞）加密代码”负样本与“真实漏洞加密代码”正样本，使 detect 层见到分布合理的两类输入，目标是让 precision 脱离“= 先验占比”的状态；同时将用途引导并入 SFT 数据（单一事实源已保证自动带入），使“训练/部署一致”。
2. **补齐五系统对照与基座基线**：用同一评估脚本对基座 qwen3-4b-instruct-2507 出通用/领域两切片数字，补全 一列，正式支撑 H1/H4。
3. **实现 ReAct Agent**：把 semgrep/rule_db/cwe_db/patch 做成工具环，让 agent 在 Confirm 前可检索规则定义与 CWE 说明，报告误报过滤收益。
4. **真实项目端到端**：对 3-5 个真实 Python 密码项目整库扫描，统计 TP/FP/FN/CWE acc，验证跨域迁移与整链价值。
5. **扩大并系统化评估**：扩充领域切片与探针谱系、加入 CWE 分布均衡的切片，避免“CWE acc 被个别类目主导”；保留训练遥测（loss/cost），把超参决策从“经验锁定”升级为数据驱动。
6. **工程化**：将“微调模型 + 规则”打包为可离线运行/可说明的审计输出，作为 CV 与后续研究的数据底座。

7 结论

通过三轮微调与提示工程迭代，本实验验证了“Semgrep 规则召回 + 领域微调 Qwen3-4B 分流 + Agent 守卫”这一混合管线在密码误用检测上的可行性：第三轮全量重训使通用切片 CWE 准确率达 0.921、领域切片 confirm recall 达 0.929；一次“只加 user 提示、不改 system”的用途引导把 18 条最难探针的误报率归零且不漏真漏洞。过程中最重要的两个结论是：（1）在

小样本下，在已微调模型上继续训练会因灾难性干扰拆掉旧任务，目标冲突时应回到基座全量重训；(2) 模型经适配后往往“能力在、行为未对齐”，正确的推理方向提示可以在不重训的前提下修复误报，但提示必须加在与训练分布一致的位置。现有模型可作为进一步研究基线，下一步以“安全/漏洞双类均衡数据”补齐 detect 判别力，并走向真实项目端到端验证。

参考文献

- [1] S-AIR-L. *PyCode-Vul*: Python Vulnerability Dataset. HuggingFace S-AIR-L/PyCode-Vul; Zenodo DOI 19746552; CC-BY-4.0.
- [2] Qwen Team. *Qwen3 Technical Report*. arXiv:2505.09388, 2025.
- [3] Hu, E. J. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021.
- [4] Dettmers, T. et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. arXiv:2305.14314, 2023.
- [5] r2c. *Semgrep*: fast, open-source static analysis. <https://semgrep.dev/>
- [6] MITRE. *Common Weakness Enumeration (CWE)*. <https://cwe.mitre.org/>
- [7] McCloskey, M. & Cohen, N. J. *Catastrophic interference in connectionist networks*. Psychology of Learning and Motivation, 1989.

A 环境与复现

编译环境：TeX Live 2026; XeLaTeX; ctex文档类; 字体 Noto Serif/Sans/Mono CJK SC。
运行环境：Python 3.10+ (requirements 固定版本); Semgrep (自定义规则回测); 阿里云百炼微调与推理 (OpenAI 兼容 HTTP, CloudBackend)。

复现命令 (与报告口径一致; 第三轮训练经百炼平台上传 data/round3/upload/ 下的 train/val/test.jsonl, 基座 qwen3-4b-instruct-2507, 超参见表 3):

```
# 领域样本规则回测 (60/60 通过)
semgrep --test rules/ <samples-dir>

# 18 探针 (同一脚本与探针, 与训前/第二轮可比)
export LLM_MODEL_ID=qwen3-4b-instruct-2507-f5d4f1aedd8f
python scripts/run_triage_probes.py --out <out.json> --compare \
    data/round2/eval/triage_probe_results_before.json

# 全量基准 (先改 configs/benchmark.yaml 的 model.model_id)
python main.py --benchmark --config configs/benchmark.yaml
```

B 关键文件与数据索引

rules/	# 10 条 CRYPTO 规则 (crypto-001-md5/rule.yaml 等) ,
CWE 见表 1	
model/prompts.py	# 提示词单一事实源 (含 TRIAGE_PURPOSE_GUIDE 用途引导)
data/round3/upload/*.jsonl	# 训练数据: train 2595 / val 130 / test 130
data/round3/eval/triage_probe_results_r3_guided.json	# 探针 (引导后)
data/round3/eval/benchmark_r3/benchmark.json	# 双切片全量基准
data/round3/eval/ACCEPTANCE.md	# 验收记录
qwen3-4b-instruct-2507-f5d4f1aedd8f	# 第三轮 (验收模型)
qwen3-4b-instruct-2507-da015253d244	# 第二轮 (受灾难性干扰)
qwen3-4b-instruct-2507-510facca98f1	# 第一轮